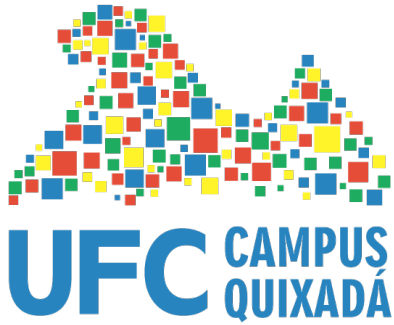




UNIVERSIDADE
FEDERAL DO CEARÁ

Aula 3 – RUP

Professora: Carla Ilane Moreira Bezerra

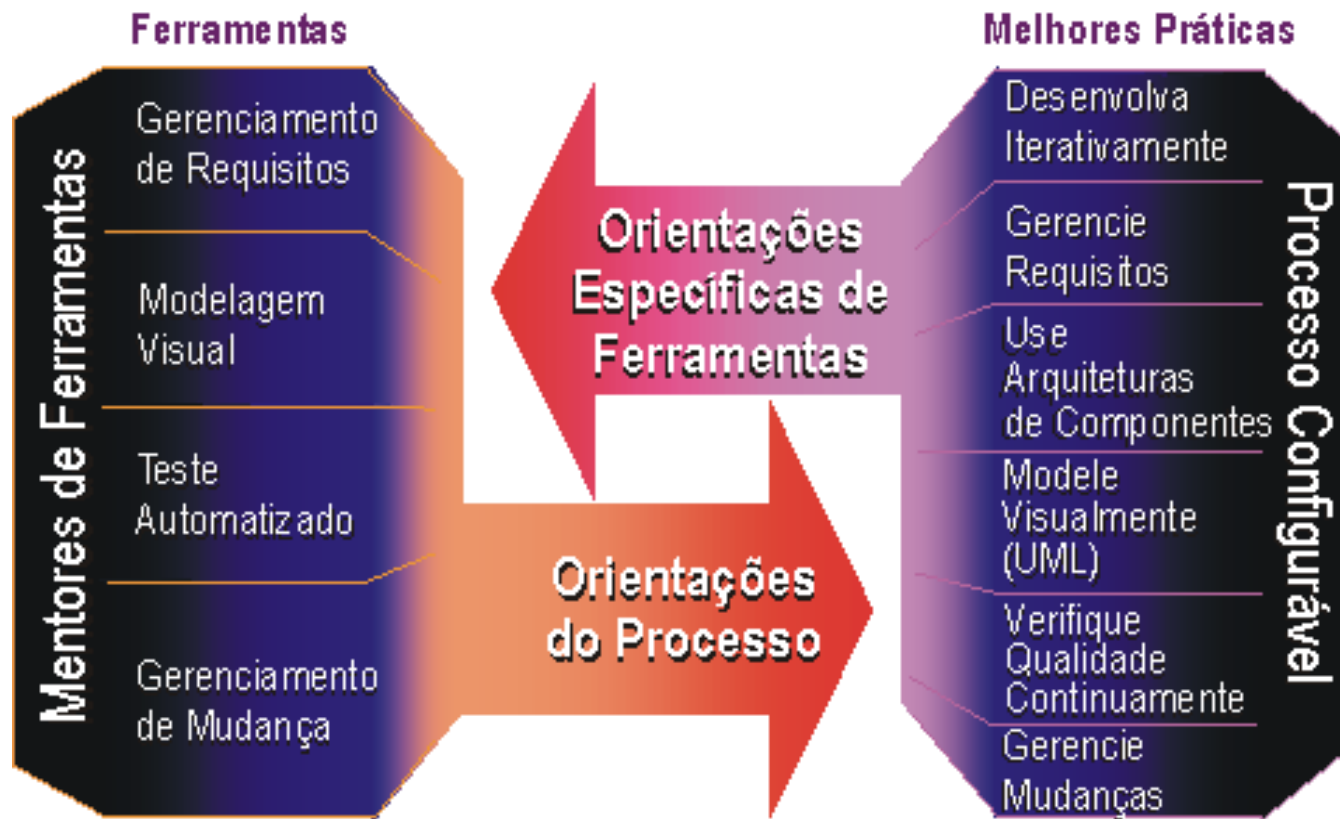
Modelo RUP

- ▶ Rational Unified Process
- ▶ Processo de engenharia de software desenvolvido pela Rational Software
- ▶ Possui um framework de processo que pode ser adaptado e estendido
- ▶ Desenvolvimento de software é feito de forma iterativa
 - ▶ Iterações planejadas em:
 - ▶ Número, duração e objetivos
- ▶ Orientado a casos de uso



Melhores Práticas

- ▶ Aplicação das melhores práticas de ES
- ▶ Uso de ferramentas para automação do processo de ES



Seis Melhores Práticas

- ▶ Desenvolver o software iterativamente
- ▶ Gerenciar requisitos
- ▶ Usar arquitetura baseada em componentes
- ▶ Modelar visualmente o software
- ▶ Verificar continuamente a qualidade do software
- ▶ Controlar as mudanças do software



Desenvolver Software Iterativamente

- ▶ Diminuição dos riscos
- ▶ Cada iteração resulta em um release executável



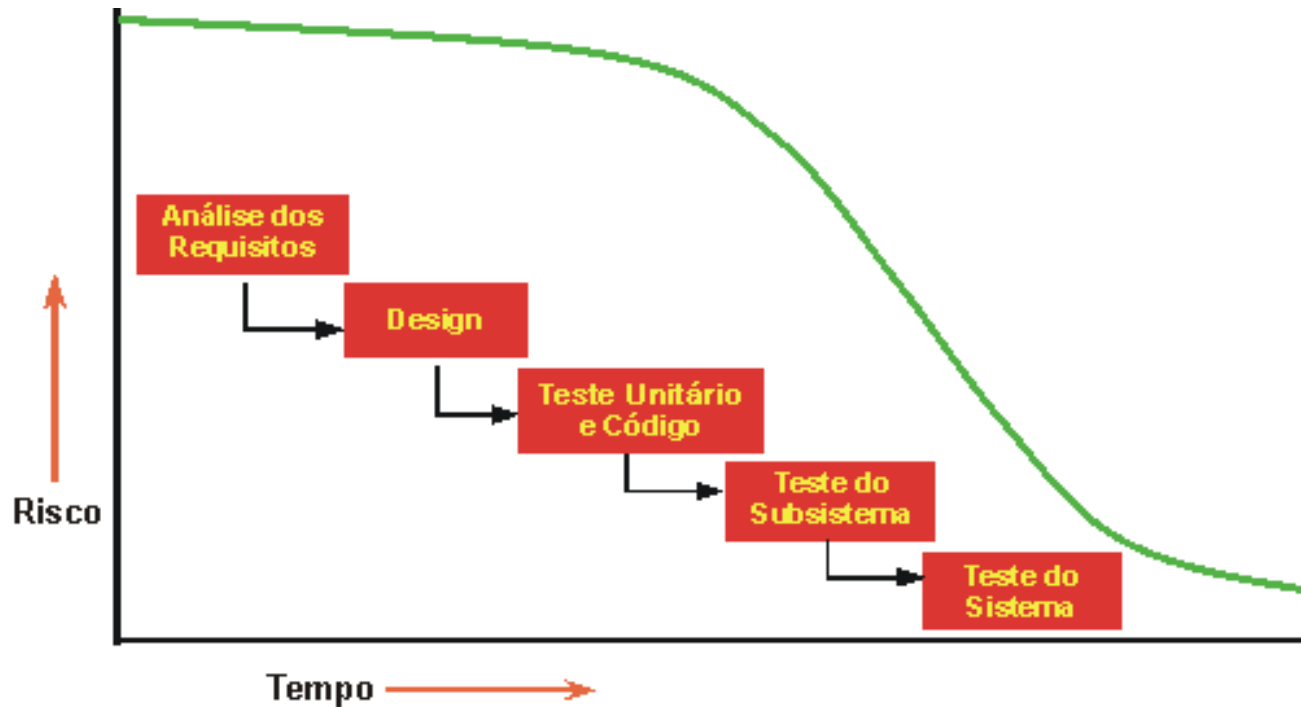
Porque Desenvolver Software Iterativamente?

- ▶ É provável que um design inicial contenha falhas em relação a seus principais requisitos
- ▶ A descoberta tardia dos defeitos de design resulta em saturações caras e, em alguns casos, até mesmo no cancelamento do projeto
- ▶ Riscos:
 - ▶ Todo projeto tem riscos
 - ▶ Quanto mais cedo identificado, mais preciso será o planejamento
 - ▶ Muitos riscos nem são descobertos até que se integre o sistema
 - ▶ É impossível prever todos os riscos



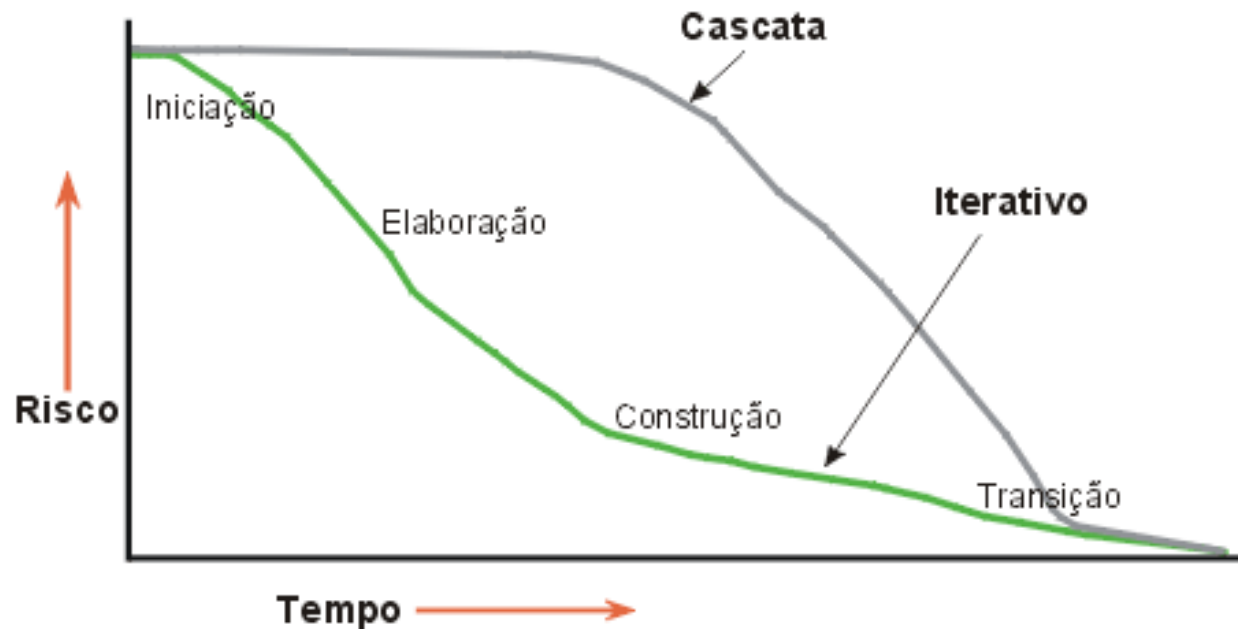
Porque Desenvolver Software Iterativamente?

- ▶ Em um ciclo de vida em cascata, você não poderá verificar se ficou livre de um risco até o final do ciclo



Porque Desenvolver Software Iterativamente?

- ▶ Em um ciclo de vida iterativo, a seleção do incremento a ser desenvolvido em uma iteração é feita com base em uma lista dos principais riscos
- ▶ Como a iteração produz um executável testado, você poderá verificar os riscos diminuíram



Vantagens

- ▶ Riscos reduzidos mais cedo, pois os elementos são integrados progressivamente
- ▶ Táticas e requisitos variáveis são acomodados
- ▶ Melhoria e o refinamento do produto são facilitados, resultando em um produto mais robusto
- ▶ Organizações podem aprender a partir dessa abordagem e melhorar seus processos
- ▶ Capacidade de reutilização aumenta



Gerenciar Requisitos

- ▶ Abordagem sistemática para:
 - ▶ Identificar
 - ▶ Localizar
 - ▶ Documentar
 - ▶ Organizar
 - ▶ Controlar
- ▶ Requisitos variáveis em um sistema
- ▶ Firmar e atualizar acordos entre o cliente e a equipe do projeto sobre os requisitos variáveis do sistema



Vantagens

- ▶ Uma abordagem disciplinada é construída no gerenciamento de requisitos
- ▶ Comunicações baseadas nos requisitos definidos
- ▶ Requisitos podem ser priorizados, filtrados e localizados
- ▶ É possível uma avaliação objetiva da funcionalidade e do desempenho
- ▶ Inconsistências são detectadas mais cedo
- ▶ Com suporte de ferramentas satisfatório, é possível fornecer um repositório para requisitos, atributos e localização de um sistema, com links automáticos para documentos externos



Arquitetura Baseada em Componentes

- ▶ **Componentes:**

- ▶ Grupos de código coesos, na forma de código fonte ou executável, com interfaces bem definidas e comportamentos que fornecem forte encapsulamento do conteúdo

- ▶ Substituíveis

- ▶ **As arquiteturas baseadas em componentes tendem a reduzir o tamanho efetivo e a complexidade da solução**

- ▶ Mais robustas e flexíveis



Arquitetura Baseada em Componentes

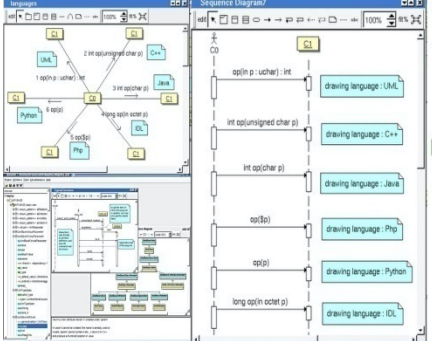
- ▶ É uma base efetiva para reutilização em larga escala
 - ▶ Ao articular claramente os principais componentes e as interfaces críticas entre eles, uma arquitetura permite que você raciocine sobre:
 - ▶ Reutilização interna
 - Identificação das partes comuns
 - ▶ Reutilização externa
 - Incorporação de componentes desenvolvidos internamente e adquiridos prontos para serem usados



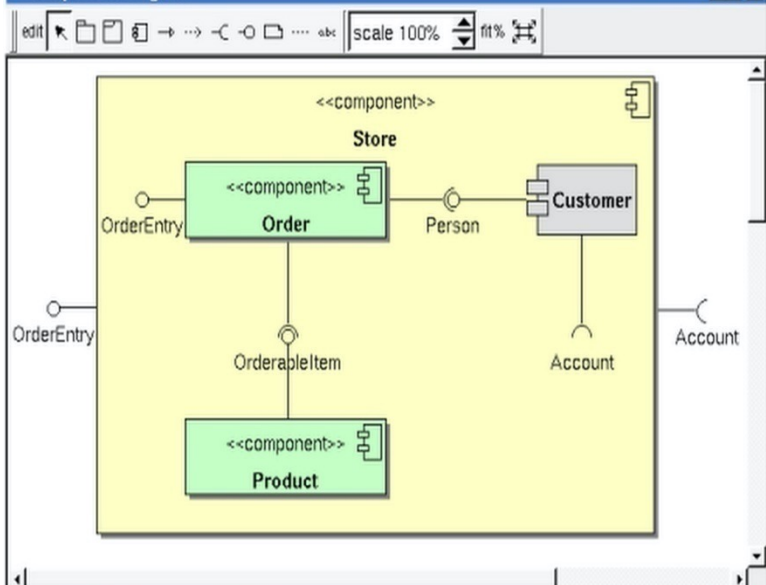
Modelar Visualmente o Software

- ▶ O que é modelagem visual ?
 - ▶ Uso de notações gráficas e textuais, semanticamente ricas, para capturar elementos de software
 - ▶ Ex: UML
 - ▶ Permite que o nível de abstração seja aumentado
 - ▶ Melhoria na comunicação
 - ▶ Permite ao leitor raciocinar sobre o modelo
 - ▶ Base não ambígua para a implementação





component diagram



language

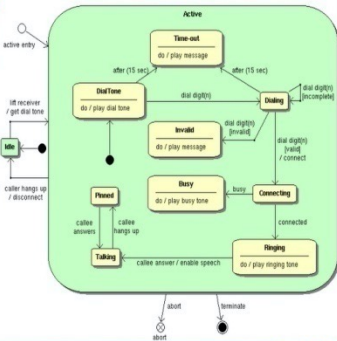
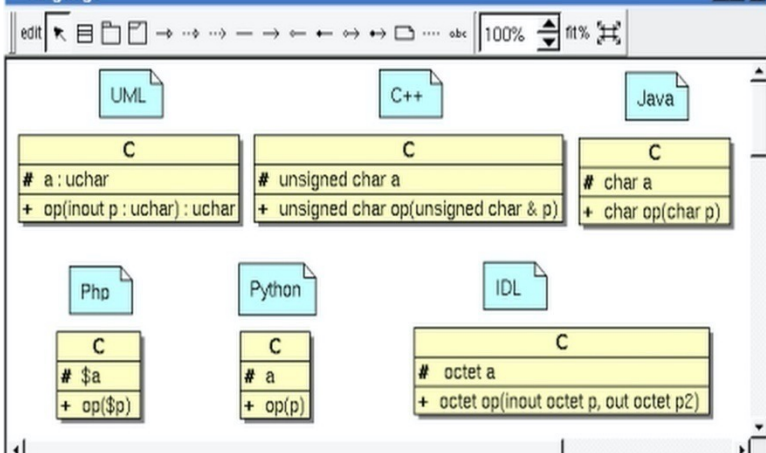
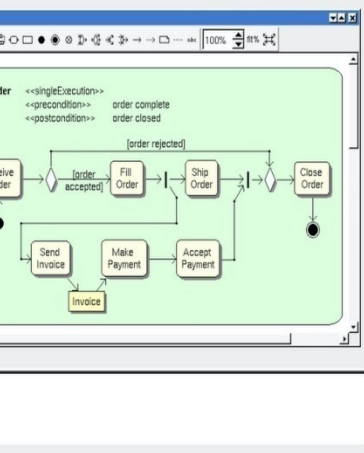
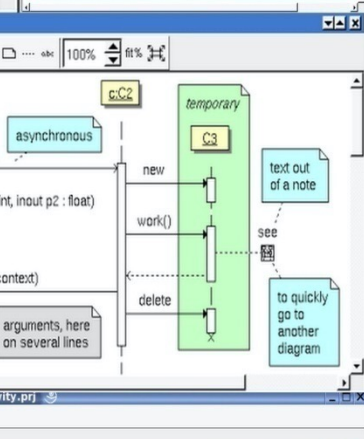
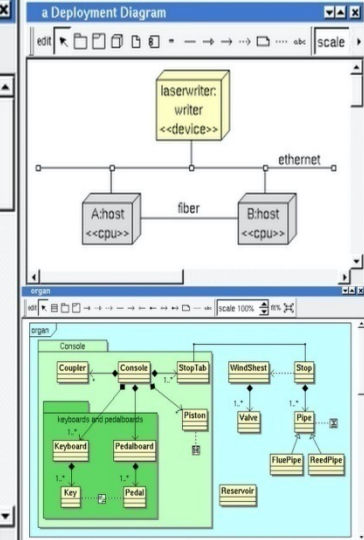
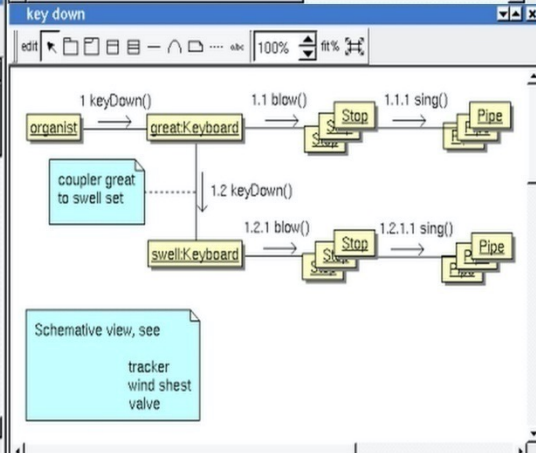
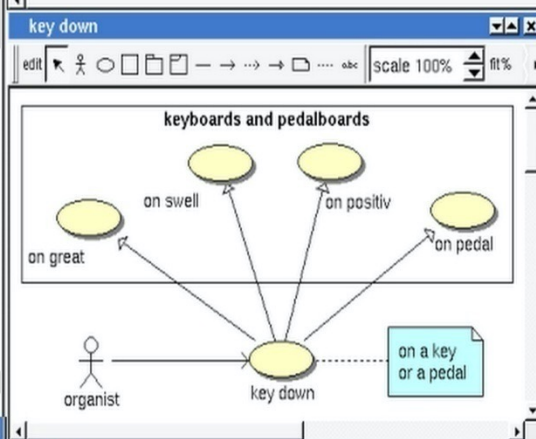
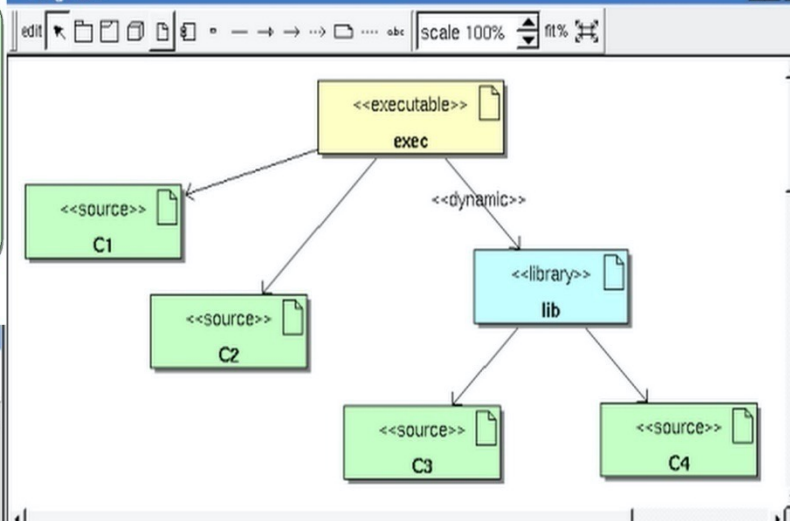


Diagram2



Verificar Continuamente a Qualidade do Software

- ▶ **Verificação da Qualidade Durante o Ciclo de Vida**
 - ▶ Avaliação da qualidade de todos os artefatos em vários pontos do ciclo de vida do projeto
 - ▶ Avaliação dos artefatos à medida que são produzidos e concluídos
 - ▶ À medida que o software executável é produzido:
 - ▶ Submissão à demonstração e teste de cenários importantes em cada iteração
 - ▶ Contraste com uma abordagem mais tradicional
 - ▶ Testes integrados do software para mais tarde



Verificar Continuamente a Qualidade do Software

▶ Qualidade do Processo e do Produto

- ▶ Qualidade não é acrescentada ao projeto por algumas pessoas
 - ▶ Responsabilidade de todos
- ▶ Qualidade do Produto
 - ▶ Qualidade do produto principal que é produzido
 - Ex: componentes, subsistemas, arquitetura
- ▶ Qualidade do Processo
 - ▶ Grau para o qual um processo aceitável foi implementado e aderiu durante a fabricação do produto
 - Ex: plano da iteração, plano de testes, realizações de caso de uso, modelos



Controlar Mudanças do Software

▶ Desafio:

- ▶ Desenvolvendo sistemas intensivos de software
- ▶ Lidar com vários desenvolvedores
- ▶ Diferentes equipes
- ▶ Diferentes locais
- ▶ Trabalhando juntos em várias iterações, releases, produtos e plataformas

▶ Ausência de controle disciplinado

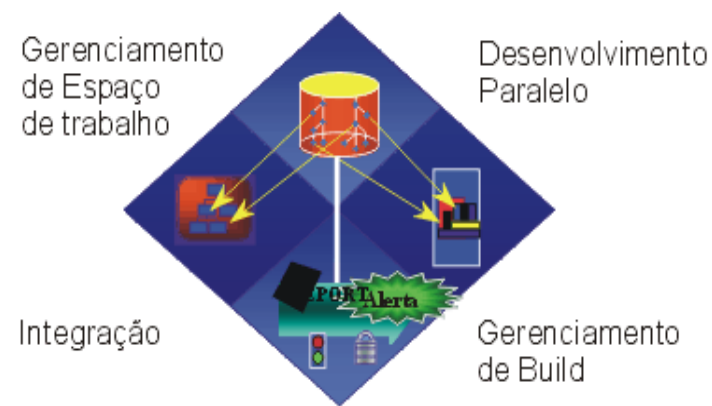
- ▶ Caos

▶ Gerência de Configuração



Controlar Mudanças do Software

- ▶ Gerenciamento de mudanças
 - ▶ Mais do que simplesmente fazer:
 - ▶ Check-in e Check-out (ClearCase)
 - ▶ Commit (CVS)
- ▶ Inclui gerenciamento de:
 - ▶ Espaços de trabalho
 - ▶ Integração
 - ▶ Builds
 - ▶ Auditorias
 - ▶ Desenvolvimento paralelo



Controlar Mudanças do Software

▶ Coordenação de Atividades e Artefatos

- ▶ Procedimentos que podem ser repetidos para o gerenciamento de mudanças e artefatos
- ▶ Permite uma melhor alocação de recursos
- ▶ Monitoramento das mudanças contínuo

▶ Coordenação de Iterações e Releases

- ▶ Estabelecimento e liberação de uma baseline testada na conclusão de cada iteração
- ▶ Manutenção da rastreabilidade entre os elementos de cada release e releases paralelos



Orientado a Casos de Uso

► Caso de Uso

- Sequência de ações executadas por um ou mais atores e pelo próprio sistema para produzir resultado de valor para um ou mais atores
- Conduzem o desenvolvimento, dirigindo as ações desde a aquisição de requisitos até a aceitação do código



Orientado a Casos de Uso

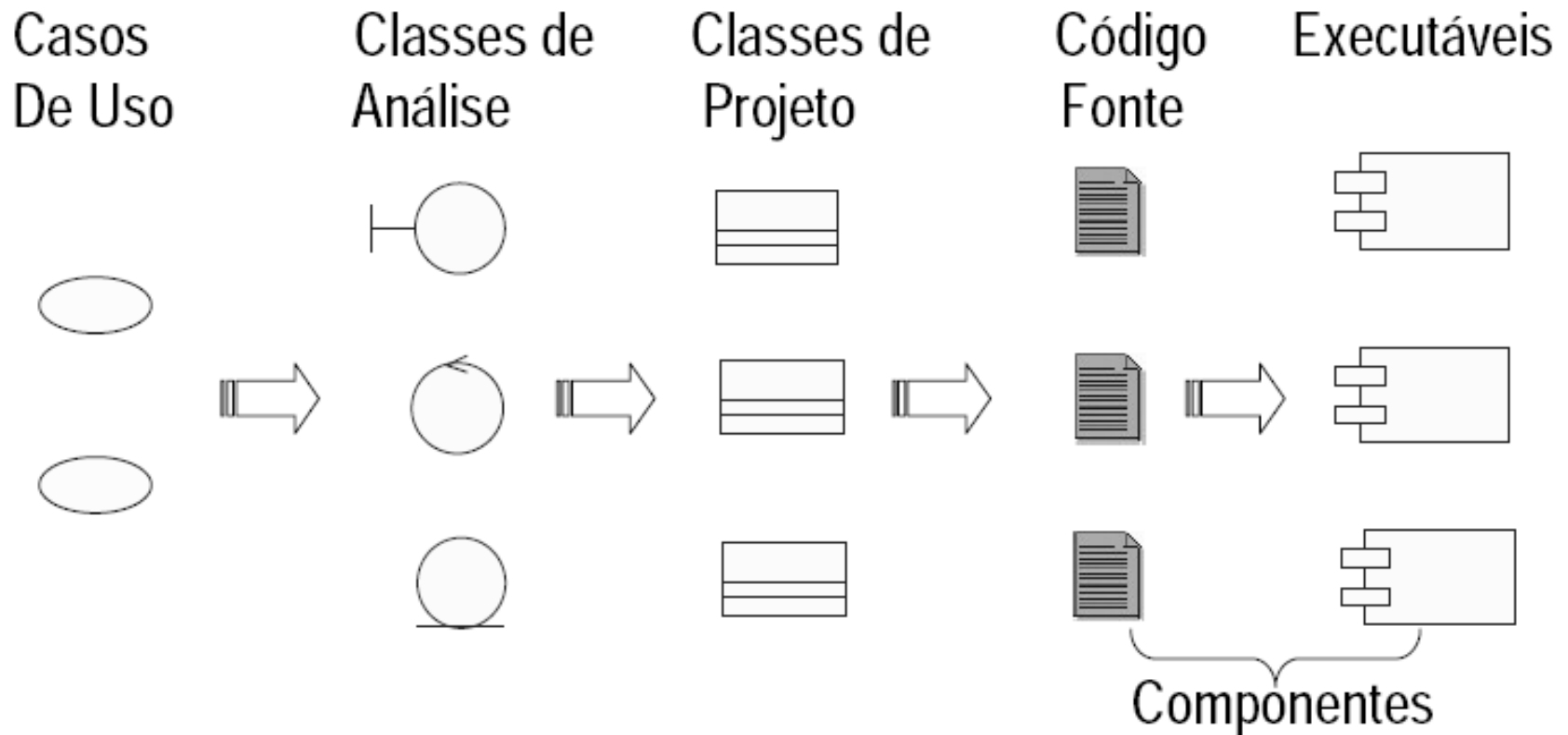
► Caso de Uso

- Adequados para capturar requisitos e dirigir análise, projeto, implementação:
 - Expressos sob a ótica dos usuários que se identificam no texto dos casos de uso
 - Fácil entendimento pelo usuário
 - Contratos entre desenvolvedores e clientes que concordam com o sistema a ser construído
 - Permitem rastreamento de requisitos a partir dos modelos posteriormente construídos a partir deles
 - Permitem decomposição dos requisitos para alocação de trabalho a equipes e facilitam a gerência de projeto



Orientado a Casos de Uso

► Base para a modelagem do sistema



Centrado na Arquitetura

▶ Arquitetura

- ▶ Visão comum que os membros da equipe devem compartilhar a fim de que o sistema produzido seja:
 - ▶ Robusto
 - ▶ Flexível
 - ▶ Escalável
 - ▶ Preço adequado



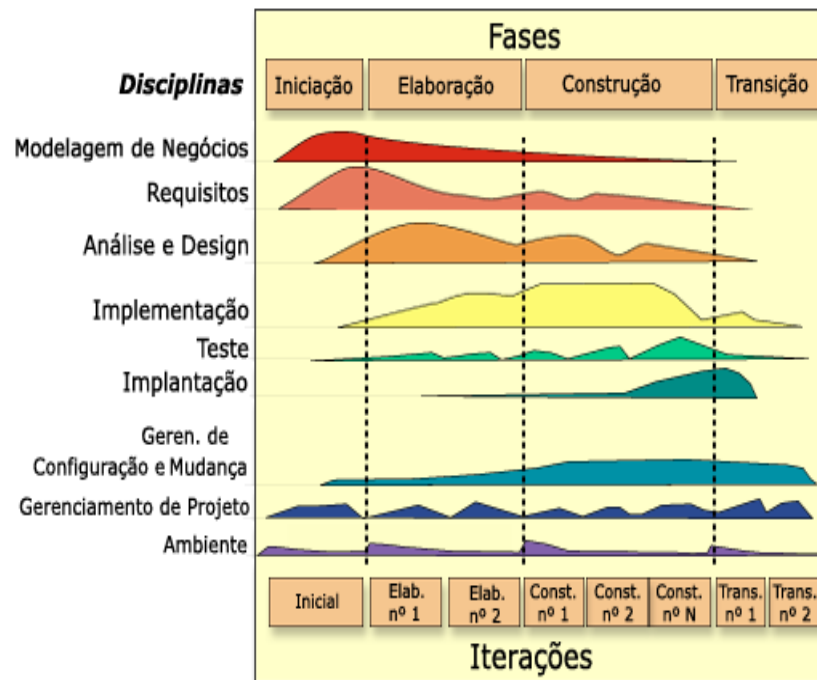
Modelo RUP

► Objetivos

- Provê um guia para as atividades da equipe de desenvolvimento
- Especifica quais artefatos devem ser desenvolvidos e quando
- Direcionam as tarefas dos desenvolvedores da equipe
- Oferece critérios para monitorar e mensurar as atividades e o produto do projeto



Rational Unified Process: Visão Geral

[Artefatos](#)
[Exemplos](#)
[Papéis](#)
[Roteiros](#)
[Mapa do Site](#)


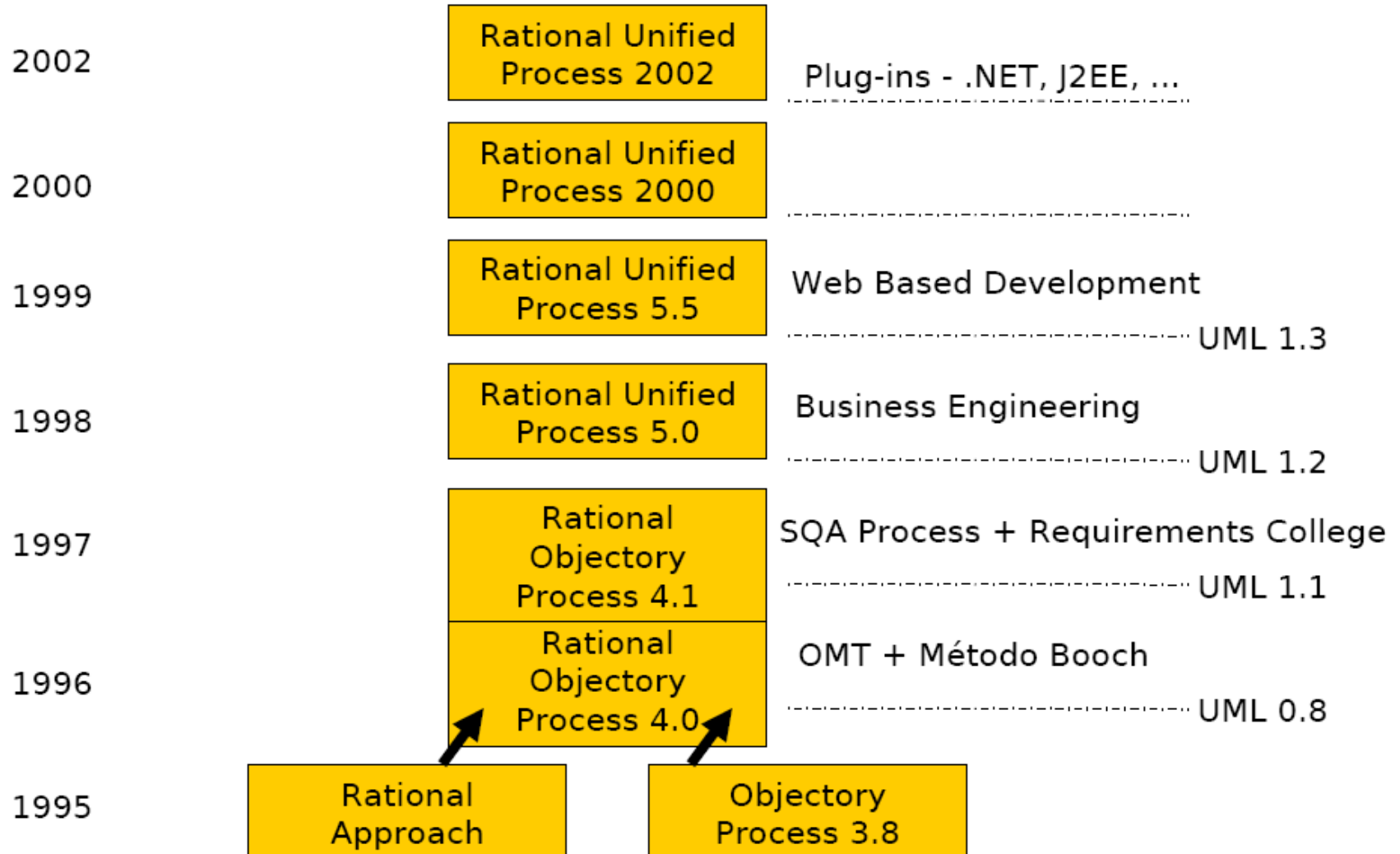
Clique em uma área da tela para obter mais informações.

O Rational Unified Process® (também chamado de processo RUP®) é um processo de engenharia de software. Ele oferece uma abordagem baseada em disciplinas para atribuir tarefas e responsabilidades dentro de uma organização de desenvolvimento. Sua meta é garantir a produção de software de alta qualidade que atenda às necessidades dos usuários dentro de um cronograma e de um orçamento previsíveis.

A figura acima mostra a arquitetura geral do RUP.

- Visão Geral
- Fases
 - Iniciação
 - Elaboração
 - Construção
 - Transição
- Disciplinas
 - Modelagem de Negócios
 - Requisitos
 - Análise e Design
 - Implementação
 - Teste
 - Implantação
 - Gerenciamento de Configuração e Muda
 - Gerenciamento de Projeto
 - Ambiente
- Papéis e Atividades
- Artefatos
- Mentores de Ferramentas
- Templates
- Artigos
- Orientações de Trabalho
- Visão Geral de Exemplos
- Kit de Ferramentas do Engenheiro de Proce
- Central de Recursos do RUP
- Sobre o RUP

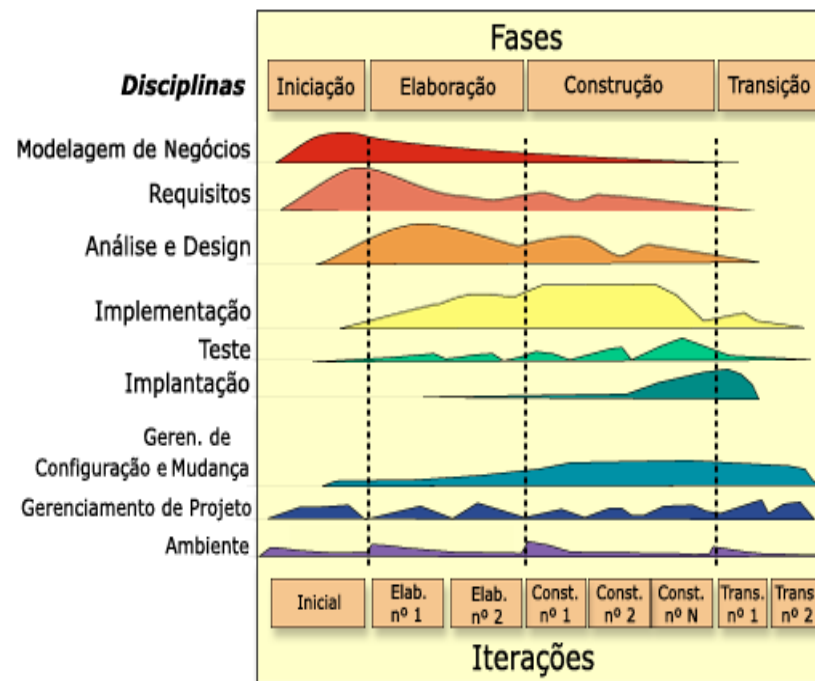
Histórico



- Visão Geral
- Fases
 - Iniciação
 - Elaboração
 - Construção
 - Transição
- Disciplinas
 - Modelagem de Negócios
 - Requisitos
 - Análise e Design
 - Implementação
 - Teste
 - Implantação
 - Gerenciamento de Configuração e Muda
 - Gerenciamento de Projeto
 - Ambiente
- Papéis e Atividades
- Artefatos
- Mentores de Ferramentas
- Templates
- Artigos
- Orientações de Trabalho
- Visão Geral de Exemplos
- Kit de Ferramentas do Engenheiro de Proce
- Central de Recursos do RUP
- Sobre o RUP

Rational Unified Process: Visão Geral

[Artefatos](#)
[Exemplos](#)
[Papéis](#)
[Roteiros](#)
[Mapa do Site](#)



Clique em uma área da tela para obter mais informações.

O Rational Unified Process® (também chamado de processo RUP®) é um processo de engenharia de software. Ele oferece uma abordagem baseada em disciplinas para atribuir tarefas e responsabilidades dentro de uma organização de desenvolvimento. Sua meta é garantir a produção de software de alta qualidade que atenda às necessidades dos usuários dentro de um cronograma e de um orçamento previsíveis.

A figura acima mostra a arquitetura geral do RUP.

Modelo RUP

- ▶ O RUP tem duas dimensões:
 - ▶ Eixo horizontal:
 - ▶ Representa o tempo e mostra os aspectos do ciclo de vida do processo à medida que se desenvolve
 - ▶ Representa o aspecto dinâmico do processo quando ele é aprovado e é expressa em termos de fases, iterações e marcos
 - ▶ Eixo vertical:
 - ▶ Representa as disciplinas, que agrupam as atividades de maneira lógica, por natureza
 - ▶ Representa o aspecto estático do processo, como ele é descrito em termos de componentes, disciplinas, atividades, fluxos de trabalho, artefatos e papéis do processo



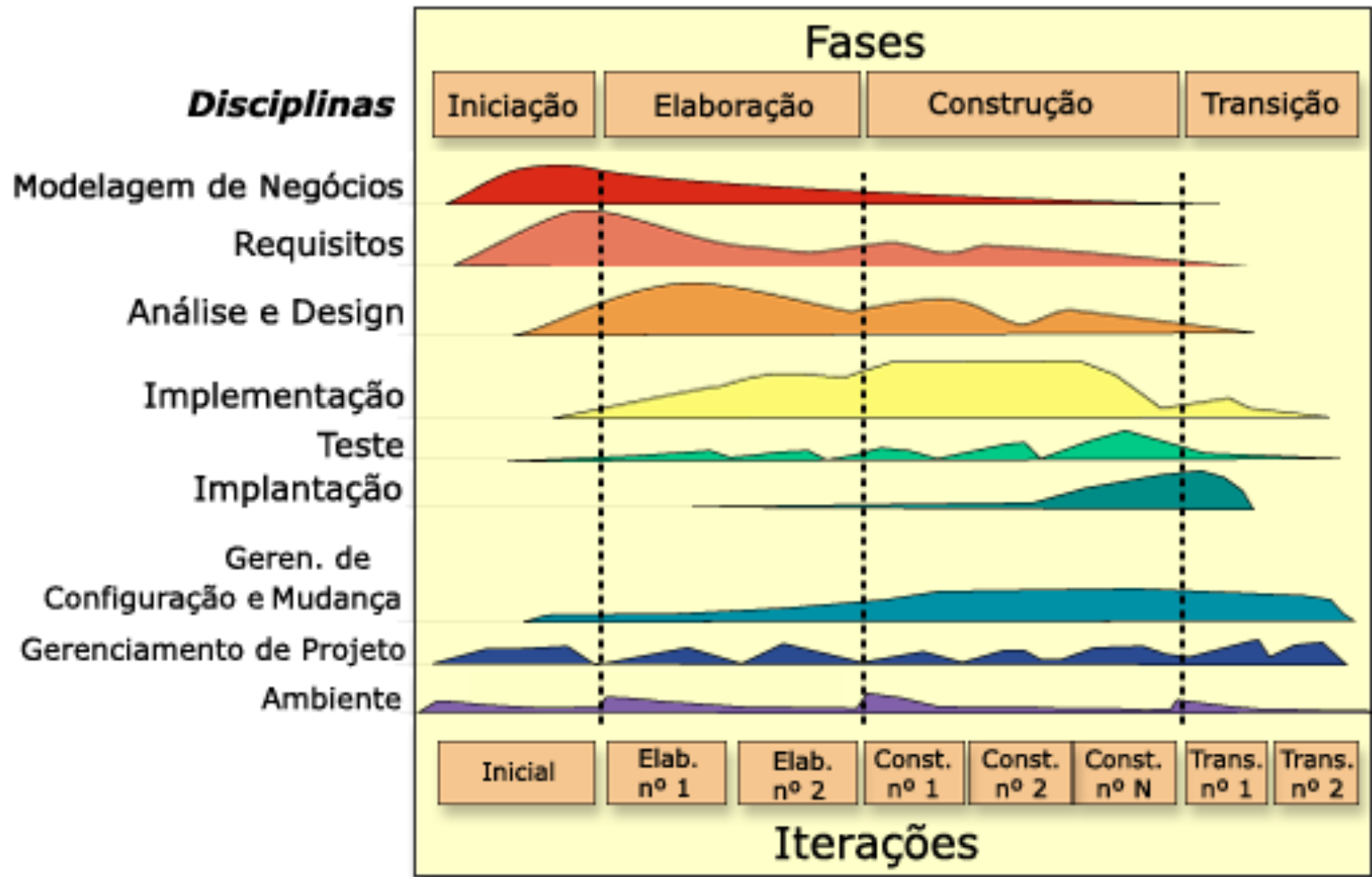
Modelo RUP

▶ Gráfico das Baleias

- ▶ Mostra como a ênfase varia através do tempo
- ▶ Exemplo
 - ▶ Nas iterações iniciais, dedicamos mais tempo aos requisitos
 - ▶ Já nas iterações posteriores, gastamos mais tempo com implementação
 - ▶ Durante todas as iterações gerenciamos projeto de forma mais ou menos uniforme

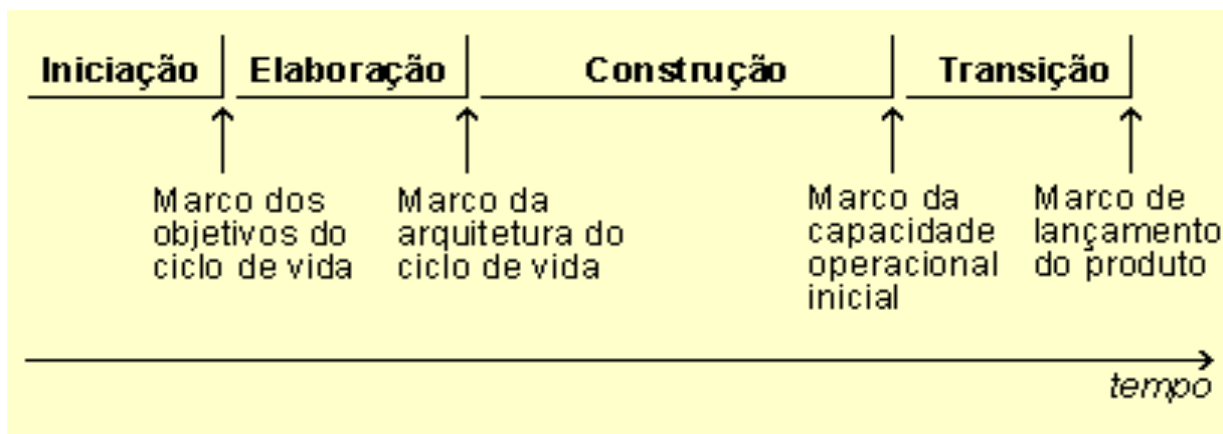


Modelo RUP



Modelo RUP

- ▶ **Concepção**
 - ▶ Ênfase no escopo do projeto
- ▶ **Elaboração**
 - ▶ Ênfase na arquitetura
- ▶ **Construção**
 - ▶ Ênfase no Desenvolvimento do sistema
- ▶ **Transição**
 - ▶ Ênfase na Implantação do sistema

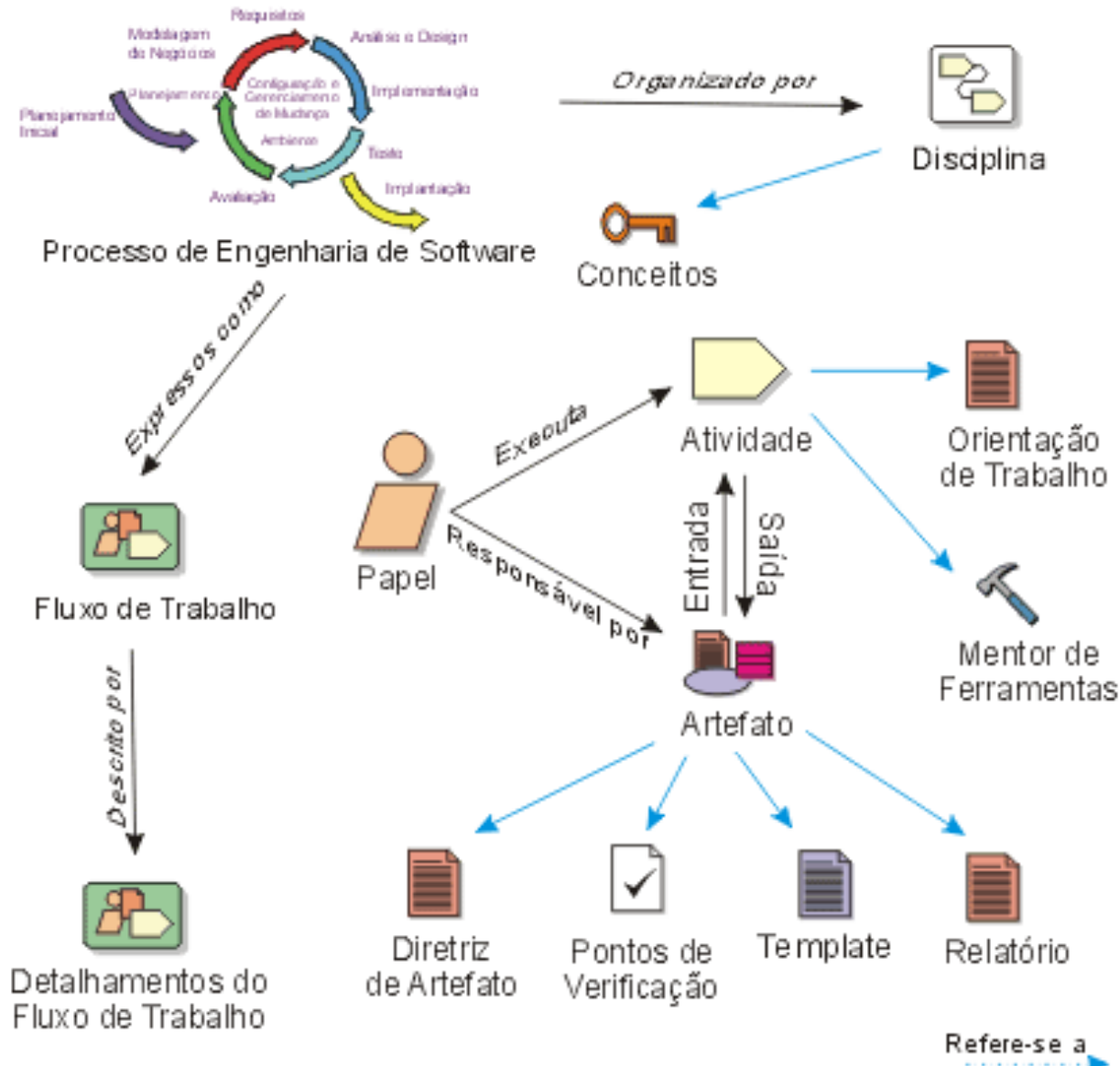


Modelo RUP

- ▶ Cada fase é dividida em iterações
- ▶ Cada fase é:
 - ▶ Planejada
 - ▶ Realiza uma seqüência de atividades distintas
 - ▶ Elicitação de requisitos
 - ▶ Análise e projeto
 - ▶ Implementação
 - ▶ Testes
 - ▶ Resulta em uma versão executável do sistema
 - ▶ Avaliada segundo critérios de sucesso previamente definidos



Conceitos Chaves



Papéis

- ▶ Papéis são desempenhados por:
 - ▶ Pessoa
 - ▶ Grupo de pessoas (equipe)
- ▶ Um membro da equipe do projeto geralmente desempenha muitos papéis distintos
- ▶ Papéis não são pessoas
 - ▶ Comportamentos
 - ▶ Responsabilidades
- ▶ Papéis Externos
 - ▶ Exemplo
 - ▶ Envolvido do projeto ou produto que está sendo desenvolvido



Atividades

- ▶ São atividades conduzidas em todas as fases de um ciclo, variando de intensidade conforme a fase
- ▶ Dão origem aos artefatos do projeto
- ▶ Em cada fase são desenvolvidas várias atividades do processo
 - ▶ Modelagem de negócio
 - ▶ Levantamento de requisitos
 - ▶ Análise e projeto
 - ▶ Implementação
 - ▶ Testes
 - ▶ ...



Papéis e Atividades

- ▶ Conjunto de atividades coerentes por eles executadas
 - ▶ Relacionadas
 - ▶ Combinadas
 - ▶ Funcionalidade



Papéis

- [Analistas](#)
- [Desenvolvedores](#)
- [Testadores](#)
- [Gerentes](#)
- [Outros papéis](#)

Artefatos



- ▶ **Produtos de trabalho**
 - ▶ Finais ou intermediários
 - ▶ Produzidos e usados durante os projetos
- ▶ **Capturam e transmitem informações do projeto**
- ▶ **Pode ser:**
 - ▶ **Documento**
 - ▶ Caso de Negócio ou Documento de Arquitetura de Software
 - ▶ **Modelo**
 - ▶ Modelo de Casos de Uso ou o Modelo de Design
 - ▶ **Elemento do modelo**
 - ▶ Classe ou um subsistema



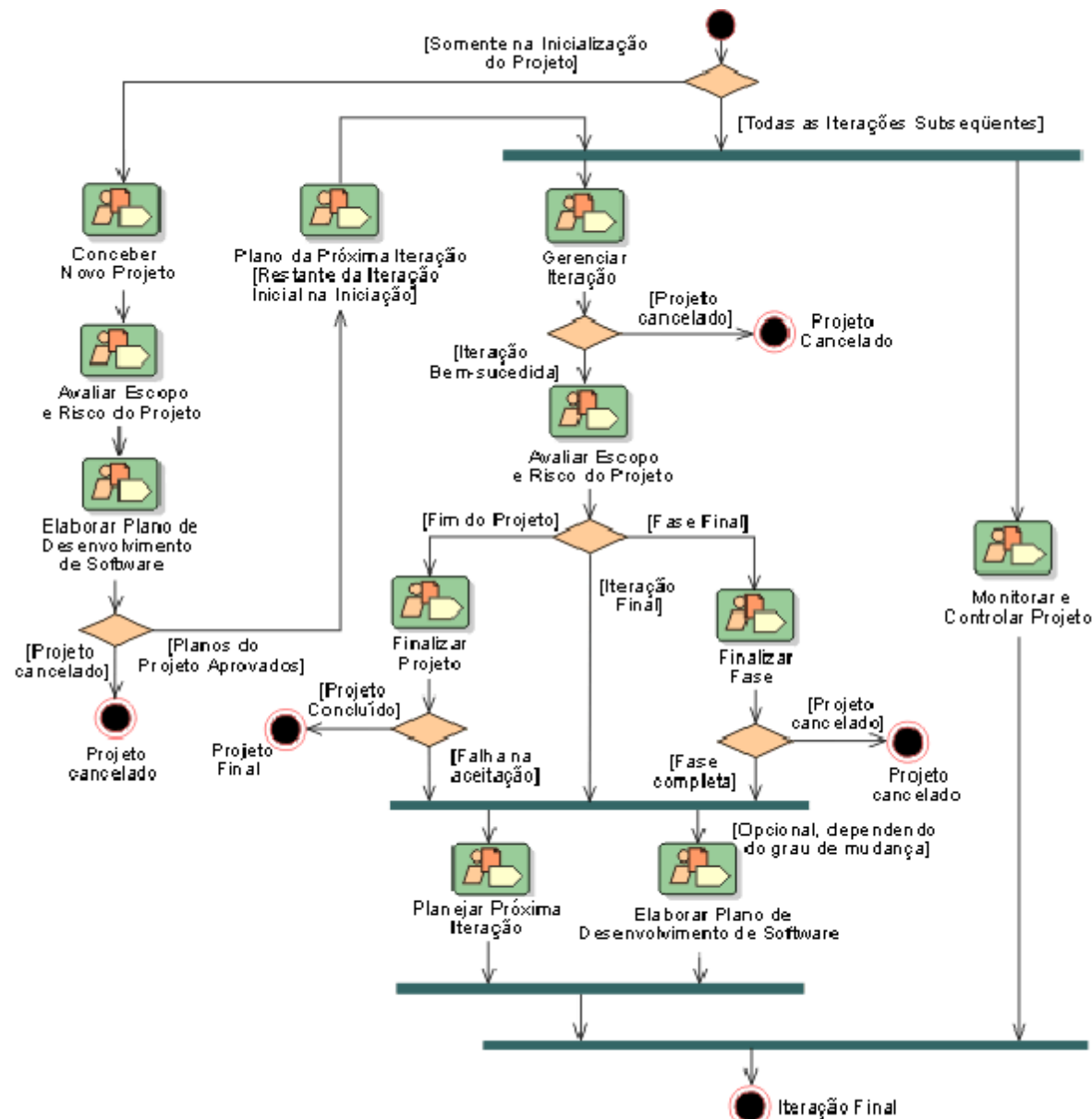
Disciplinas

- ▶ Mostra todas as atividades que devem ser realizadas para produzir um determinado conjunto de artefatos
- ▶ Nível geral
 - ▶ Resumo de todos os papéis, atividades e artefatos envolvidos
- ▶ Nível detalhado
 - ▶ Colaboração entre papéis
 - ▶ Utilização e produção de artefatos



Disciplinas

► Gerenciamento de Projetos



Vantagens do RUP

▶ Vantagens:

- ▶ Permite e encoraja o feedback do usuário, elicitando os requisitos reais do sistema
- ▶ Inconsistências entre requisitos, projeto e implementação podem ser detectados rapidamente
- ▶ Divisão da carga de trabalho por todo o ciclo de vida
- ▶ Compartilhamento de lições aprendidas, melhorando continuamente o processo
- ▶ Evidências concretas do andamento do projeto podem ser oferecidas durante todo o ciclo de vida



Referências Bibliográficas

- <http://www.wthreex.com/rup/portugues/index.htm>
- R.S. Pressman, Engenharia de Software, Rio de Janeiro: McGraw Hill, 5ª edição, 2002
- I. Sommerville, Engenharia de Software, São Paulo: Addison-Wesley, 9ª edição, 2011
- Engenharia de Software - Notas de Aula - Ricardo de Almeida Falbo - UFES - Universidade Federal do Espírito Santo
- Introdução à Engenharia de Software e Modelos de Processos de Software - Engenharia de Software - Profa. Inês A. G. Boaventura
- Pós Graduação - Engenharia de Software - Ana Candida Natali - COPPE/UFRJ - Programa de Engenharia de Sistemas e Computação - FAPEC / FAT
- Princípios de Análise e Projeto Orientados a Objetos com UML - Prof. Renata Rodrigues de Ávila - Adaptação das Notas de Aula do Prof. Eduardo Bezerra - Editora CAMPUS
- FACULDADE QI - Graduação Tecnológica em Desenvolvimento de Sistemas - Disciplina de Engenharia de Software - André Tocchetto
- Introdução à Engenharia de Software e Modelos de Processos de Software - Engenharia de Software - Profa. Inês A.G.Boaventura
- Engenharia de Software II - Bianca Zadrozny - <http://www.ic.uff.br/~bianca/engsoft2/>
- Modelos de Processo de Software - Escola Técnica Federal de Palmas - Análise e Projeto de Sistemas Orientados a Objetos - Prof. Gerson P. Focking



Dúvidas?



UNIVERSIDADE
FEDERAL DO CEARÁ

